# Memory, Context Engineering & Prompt Caching

MSc. Ngô Ngọc Đăng Khoa

MEMORY, CONTEXT & CACHING

# AGENDA

01 Problems

02 Persistent memory

03 Compaction và Code Mode

04 Prompt caching và model switching

# Problems

**Remember**: điều gì phải còn khi sang session mới?

1

**Manage**: context chứa gì sau hàng trăm tool call?

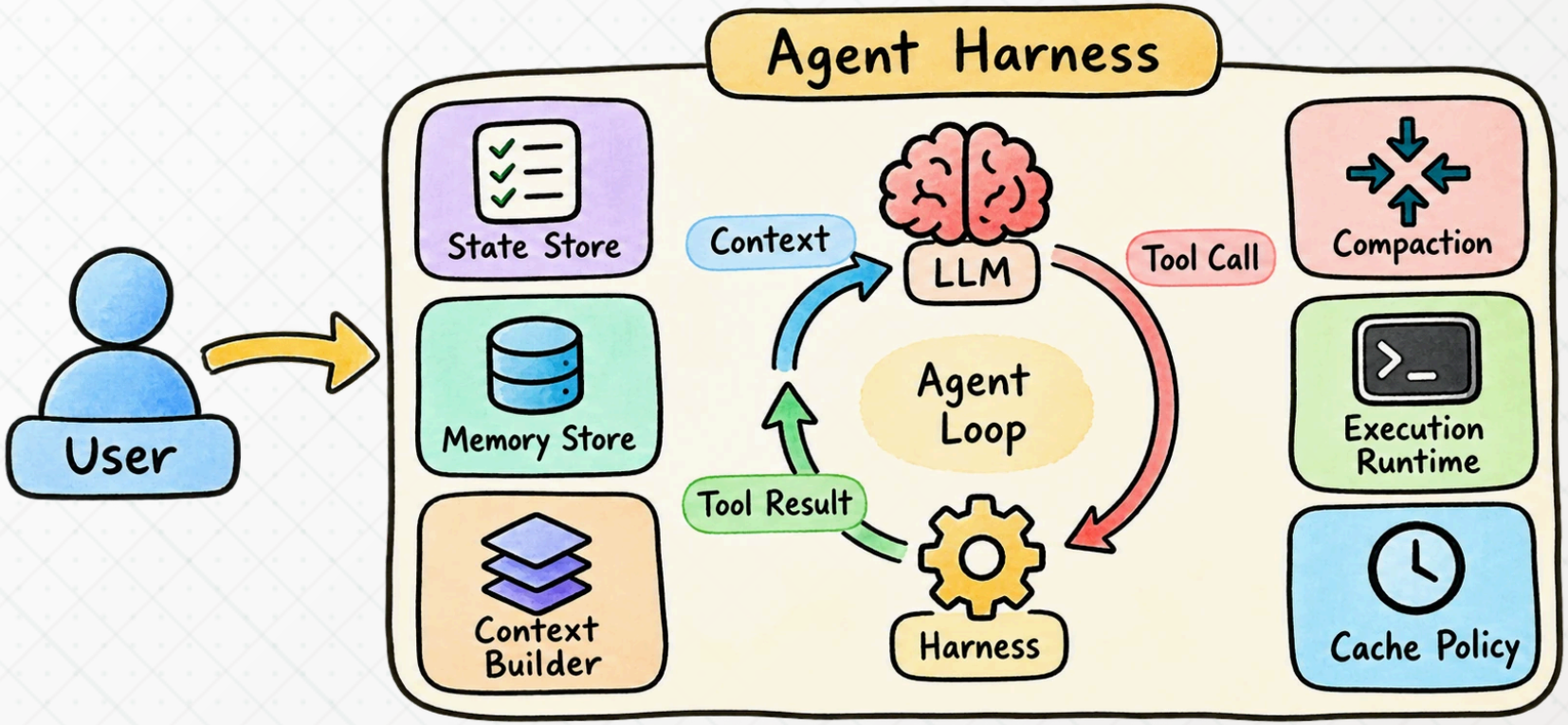
2

**Reuse**: phần tính toán nào dùng lại được qua hàng trăm LLM call?

3

# Từ agent loop đến agent harness

> **Agent loop** là lõi điều phối; **harness** là hệ thống vận hành xung quanh lõi đó.



# Một tình huống kéo dài nhiều ngày

- **Ngày 1**: thống nhất stack PostgreSQL, TypeScript và quy ước API
- **Ngày 2**: sửa authentication, thử ba cách, hai cách thất bại
- **Ngày 3**: user yêu cầu tiếp tục trên một session mới
- Hàng trăm file và log đã được đọc trong quá trình làm

> **Là người xây dựng harness, bạn giữ lại gì và đưa gì vào LLM?**

PHẦN I

# Persistent memory

Memory không phải một transcript chat vô hạn.

# LLM: stateless, agent: stateful

> Một lần gọi API chỉ thấy **dữ liệu của request hiện tại**.

- ☕ Harness quyết định **lưu** thông tin nào
- ☕ Harness quyết định **truy xuất** thông tin nào
- ☕ Harness quyết định **đưa lại** gì vào request kế tiếp

| Thành phần | Trả lời câu hỏi | Ví dụ |
| --- | --- | --- |
| **State** | Đang làm đến đâu? | `current_step=run_tests` |
| **History** | Đã có tương tác gì? | User và tool messages |
| **Memory** | Điều gì cần nhớ? | Project dùng PostgreSQL |
| **Context** | LLM thấy gì *lúc này*? | Rules, memory đã chọn và task |

**Memory được lưu trên ổ đĩa không tự nhiên xuất hiện trong context: cần bước `retrieval` và `injection`.**

# Phân loại memory

**Session memory**: mục tiêu, tiến độ, lỗi chưa xử lý, bước kế tiếp

**Persistent memory**: sở thích user, quy ước project, quyết định, bài học

**Instructions và policies**: rules bắt buộc, có phạm vi và độ ưu tiên riêng

Không phải dòng chat nào cũng là memory.

# Case study: AGENTS.md và MEMORY.md

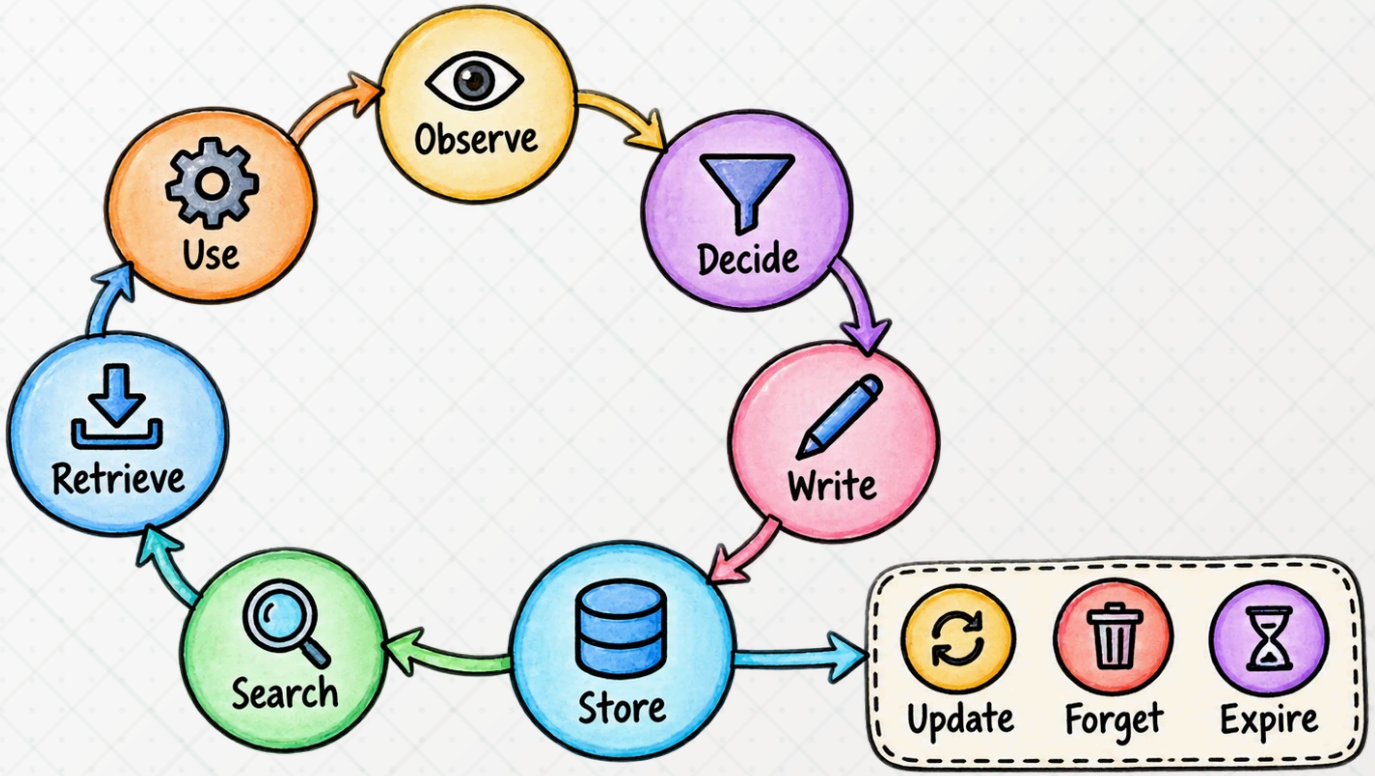
| File | Vai trò |
| --- | --- |
| `project/AGENTS.md` | Quy ước project |
| `~/.claude/AGENTS.md` | Hướng dẫn theo user |
| `MEMORY.md` | Index ngắn |
| `memory/auth.md` | Chi tiết authentication |
| `memory/preferences.md` | Sở thích lập trình |

**Index nhỏ** luôn được nạp; chi tiết chỉ nạp khi cần.

Nhồi tất cả memory mỗi lượt: *context bloat* quay lại

# Memory lifecycle

> Memory cần chính sách **ghi, đọc, sửa và quên**.



# Memory sai và lỗi thời

> Memory cần **giải quyết xung đột**, không chỉ append.

## Hai session

- **Session 1**: "Authentication sử dụng JWT."
- **Session 2**: "Chuyển sang session cookie."

## Lựa chọn nào hợp lý?

- Giữ cả hai như quy định hiện hành
- Xóa hoàn toàn dấu vết JWT
- Đánh dấu JWT *đã bị thay thế*, lưu nguồn và thời điểm

# Memory curation

| Mẫu kiến trúc | Đặc điểm |
| --- | --- |
| Markdown instructions | Đơn giản, dễ bắt đầu |
| Memory tools | Agent quyết định lúc nào đọc, ghi |
| Progressive disclosure | Đọc index trước, load chi tiết sau |
| File-system, searchable memory | Tổ chức và truy xuất linh hoạt |
| Out-of-band curation | Phân tích nhiều session để cập nhật memory |

# Challenge A: persistent memory

> Mục tiêu: agent **nhớ được** quy ước dự án qua nhiều session, và cập nhật khi quy ước thay đổi.

- ☕ Có thể dùng lại agent đã có
- ☕ **Chứng minh 1**: session mới vẫn dùng đúng quy ước đã thống nhất
- ☕ **Chứng minh 2**: quy ước đổi thì các session sau dùng quy ước mới

**Timebox**: 30 phút.

# Challenge A: persistent memory

## Gợi ý

- **Lưu ở đâu**: file Markdown, JSON hay SQLite là đủ?
- **Ai ghi**: agent tự gọi tool ghi memory, hay harness tự trích từ hội thoại?
- **Khi nào nạp**: memory vào context lúc mở session hay khi cần mới tìm?
- **Khi quy ước đổi**: xem lại slide Memory sai và lỗi thời
- **Kiểm chứng**: in ra phần memory đã được inject vào context

# So sánh thiết kế memory

| Thiết kế | Điểm mạnh | Đổi lại |
| --- | --- | --- |
| JSON, Markdown | Dễ làm, dễ kiểm tra | Khó tìm kiếm và xử lý xung đột khi lớn |
| SQLite và metadata | Truy vấn, update rõ | Cần schema và migration |
| Vector retrieval | Tìm theo ngữ nghĩa | Chi phí embedding, false matches |
| Index và topic files | Lazy load, dễ đọc | Phải giữ index chính xác |

PHẦN II

# Giữ context nhỏ gọn

# Context window

> Context window là **working set**, không phải kho dữ liệu vô hạn.

- Gồm instructions, memory đã chọn, history, tool results, dữ liệu retrieve
- Phải chừa chỗ cho **output** của model
- Nhiều dữ liệu hơn *không* đồng nghĩa quyết định tốt hơn

# Tool-output bloat

> Raw output có thể được **archive**, không cần nằm mãi trong active context.

## Raw output đã nhận

- `read_file`: **2.000** dòng
- `run_tests`: **7.000** dòng
- `grep`: **1.500** dòng, cộng log retry lặp lại

## Agent thực sự cần

- **3** test đang lỗi
- Nguyên nhân khả dĩ
- Các file liên quan
- Thử nghiệm kế tiếp

# Compaction

- Mục tiêu *không phải* kể lại mọi điều đã xảy ra
- Mục tiêu là thu thập đủ thông tin cho **hành động tiếp theo**
- Agent phải tiếp tục công việc một cách chính xác

> Compaction là **checkpoint**, không phải bản recap của chat.

# Compaction toolbox

| Kỹ thuật | Cách làm | Rủi ro |
|---|---|---|
| Truncation | Bỏ messages cũ | Mất ràng buộc |
| Tool-output pruning | Loại log thô hoặc trùng lặp | Bỏ sót bằng chứng |
| Summary | LLM rút gọn history | Hallucinate, bỏ sót |
| Structured checkpoint | Lưu state theo schema | Thiếu trường quan trọng |
| Incremental summary | Nén từng giai đoạn | Lỗi tích lũy |
| Selective keep/drop | Giữ nguyên các đoạn quan trọng | Chọn nhầm đoạn |

# Structured checkpoint

```
goal: Fix refresh-token race condition
current_state: handler implemented; test failing
constraints: keep API backward compatible
findings:
  - failure occurs during concurrent refresh
failed_attempts:
  - increasing token TTL did not help
files: [auth.ts, session.ts, auth.test.ts]
next_action: inspect locking around refresh
```

- ☕ Kiểm chứng checkpoint bằng cách cho agent **tiếp tục task** khác

# Selective compaction

Phần nào cần giữ trong context window

**KEEP**: quyết định thiết kế, lỗi chưa giải quyết, yêu cầu bắt buộc

1

**TRUNCATE**: output test dài rút còn vài dòng lỗi chính

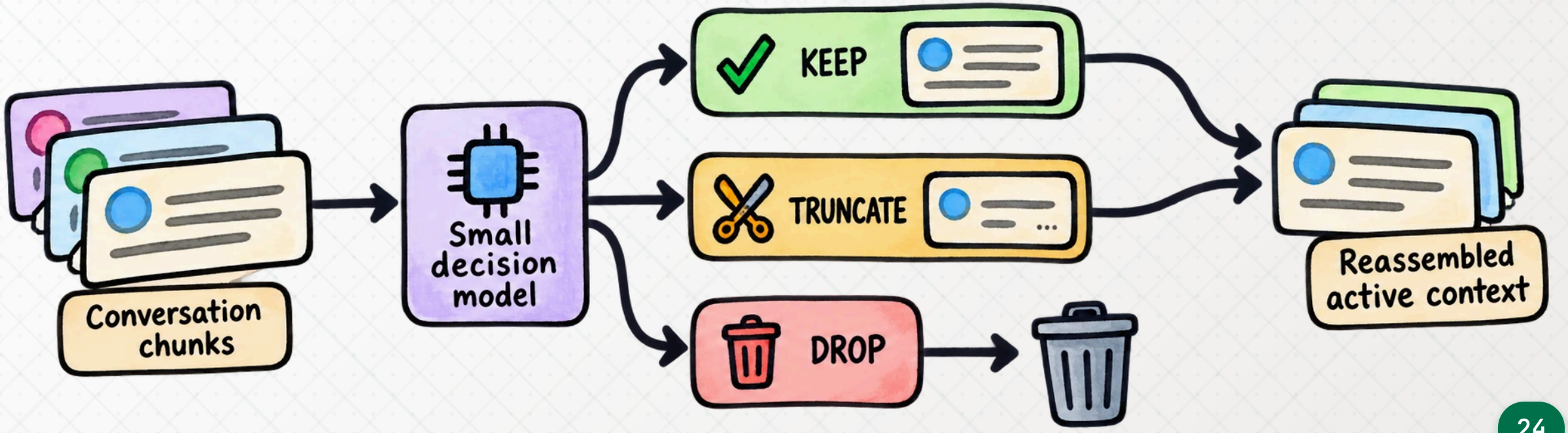
2

**DROP**: log trùng, thử nghiệm thất bại đã kết luận, dữ liệu dễ tái tạo

3

# Jev và System One Model

> Giữ chính xác khi cần; nén khi có thể; **loại khi an toàn**.



# Thời điểm compact

**Context utilization** vượt ngưỡng an toàn

Tool outputs và history **lặp lại** quá nhiều

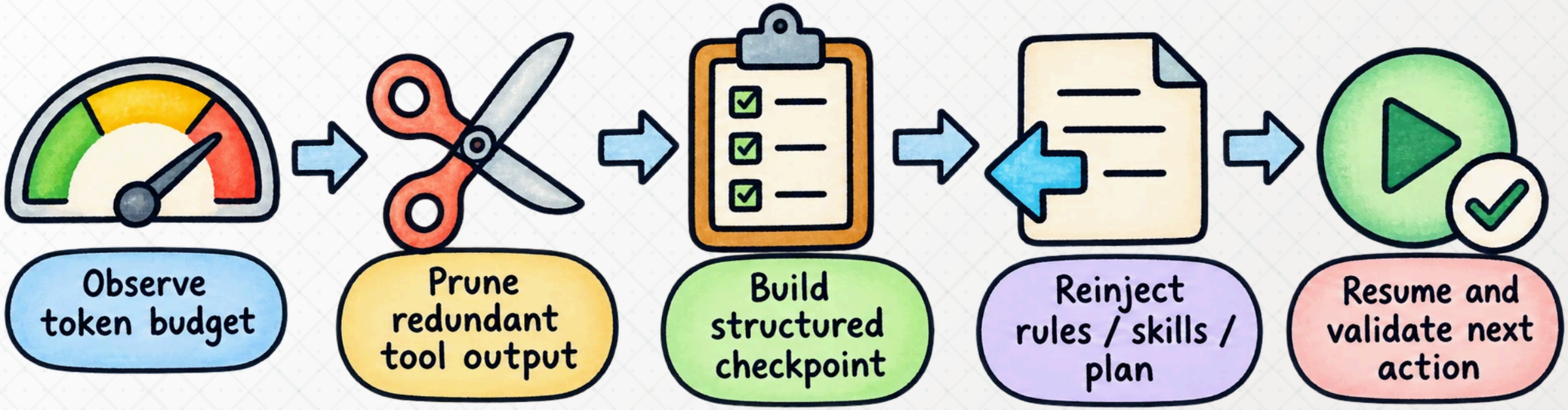
Kết thúc một **milestone**, cần checkpoint

Trước khi **cold-start** lại một context rất lớn

**Không đợi context đầy hẳn rồi mới compact.**

# Compaction pipeline

> Compaction có thể xem như một **controlled reboot** của working context.



# Code Mode

# Code Mode

1

**Viết**: Model trả tool call `run_code` kèm chương trình.

2

**Chạy**: Runtime chạy chương trình, chương trình gọi tools.

3

**Nhận**: Model chỉ nhận kết quả đã xử lý.

# Native tool calling và Code Mode

> Các tool vẫn giữ nguyên; điểm khác là **chương trình** quyết định chuỗi lời gọi tool thay cho LLM.

| Native tool calling | Code Mode |
| --- | --- |
| LLM trả tool call `read_file` | LLM trả tool call `run_code` |
| Tool output trả về LLM | Code nhận tool output |
| LLM yêu cầu file tiếp theo | Code tự lặp và aggregate |
| LLM xử lý raw results | LLM nhận findings |

# Ví dụ Code Mode

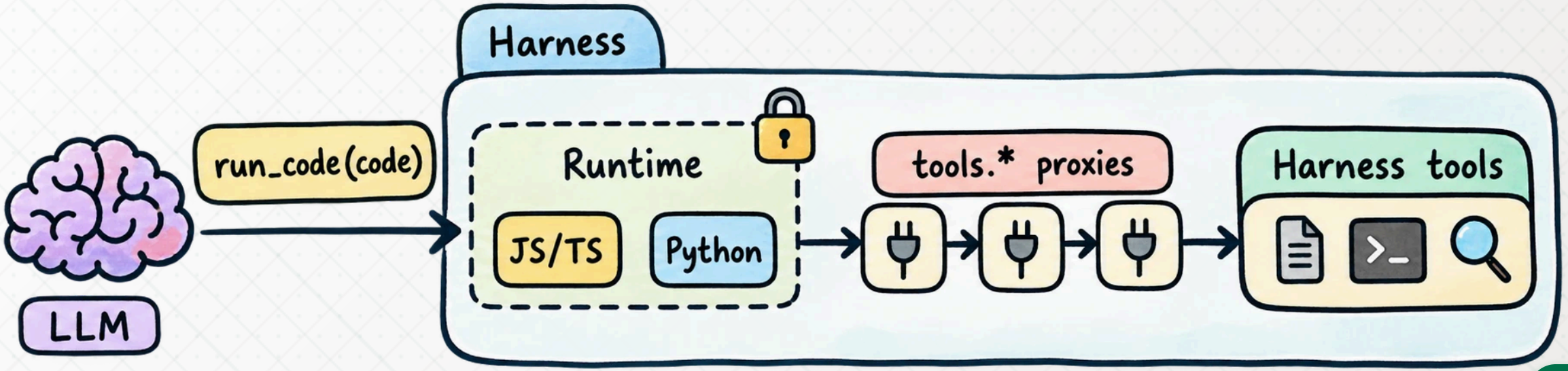
```
const files = await tools.list_files("src/**/*.ts");
const hits = [];
for (const path of files) {
  const content = await tools.read_file(path);
  const count = (content.match(/TODO/g) ?? []).length;
  if (count) hits.push({ path, count });
}
return hits.sort((a, b) ⇒ b.count - a.count).slice(0, 5);
```

- **100 file** được đọc trong runtime, chỉ **5 findings** đi vào LLM
- `tools.*` là SDK minh họa; LLM biết API qua tài liệu hoặc discovery

# Runtime của Code Mode

> Chạy code trong runtime riêng *chưa phải* sandbox bảo mật; vẫn phải **giới hạn quyền** truy cập.

☕ **Production**: sandbox, permission check, timeout, resource limit



# Challenge B: context management

> Mục tiêu: giữ context **nhỏ gọn** mà agent vẫn làm đúng việc.

**Timebox**: 30 phút, 15 phút mỗi bài.

## B1: Compact

- Session dài được compact
- Agent vẫn nhớ lỗi đang điều tra, cách đã thử, bước tiếp

## B2: Code Mode

- Agent trả lời câu hỏi trên nhiều file log
- Raw log *không* đi vào context

# Challenge B: context management

## Gợi ý

### B1: Compact

- ☕ Tạo session dài có log lớn để thử
- ☕ Khi nào kích hoạt compact?
- ☕ Giữ gì, bỏ gì: xem Selective compaction

### B2: Code Mode

- ☕ Có tool chạy lệnh: code chạy ở đâu?
- ☕ Script chỉ in kết quả cuối
- ☕ Đo input tokens qua `usage`

# Compaction và Code Mode

> Hai kỹ thuật **bổ sung** cho nhau, không đối lập.

| Tiêu chí | Compaction | Code Mode |
|---|---|---|
| Thời điểm | Sau khi dữ liệu tích lũy | Khi chạy tools |
| Mục tiêu | Rút gọn working context | Tránh intermediate outputs |
| Rủi ro | Mất state, constraints | Code lỗi, quyền truy cập |
| Kết hợp | Checkpoint cho long session | Giảm tốc độ context growth |

PHẦN III

# Prompt caching và model switching

# Prompt caching và response caching

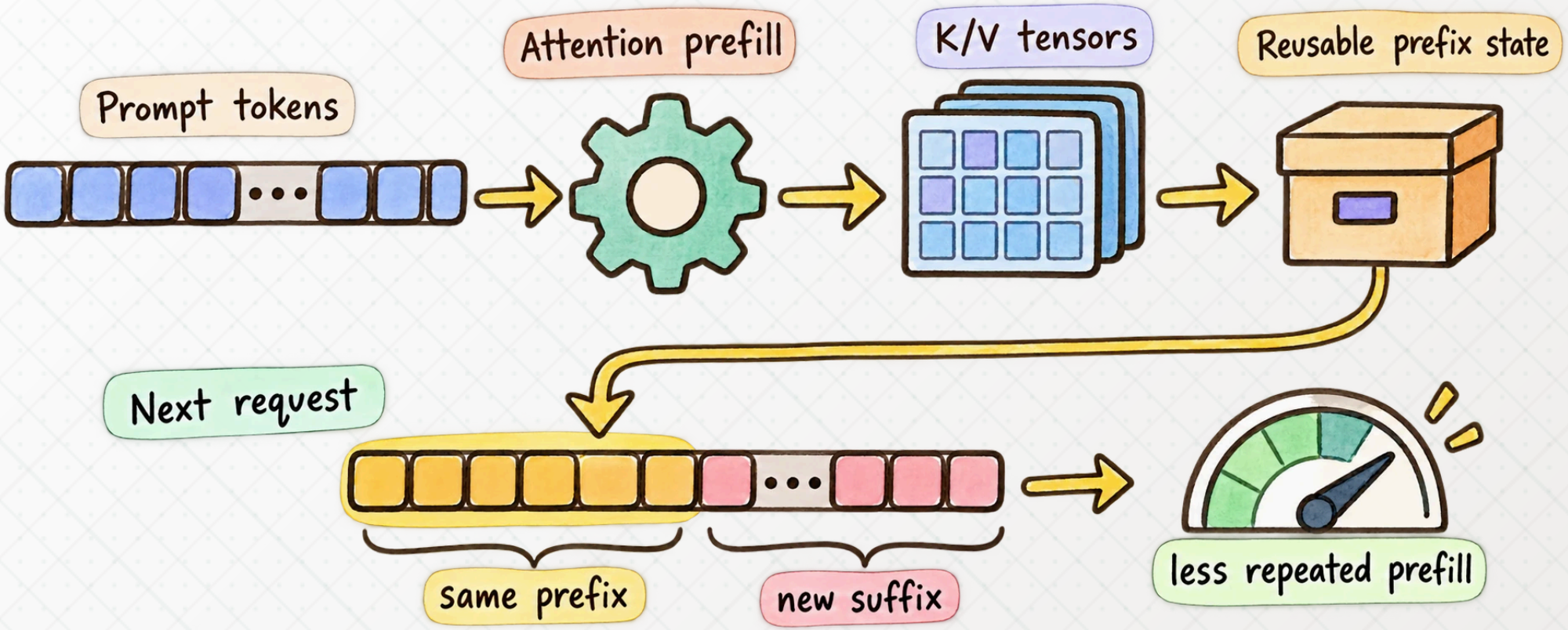
Prompt caching *không phải* response caching.

**Response cache**: trả lại answer cũ gặp lại câu hỏi cũ

**Prompt cache**: dùng lại tính toán của input khi prefix trùng khớp

# KV cache

> Cached tokens *vẫn* nằm trong input context; chỉ rẻ và nhanh hơn khi **cache hit**.



# Input cost tăng theo history

| Lượt gọi | Input tokens |
|---|---|
| Turn 1 | 51k |
| Turn 2 | 55k |
| Turn 3 | 59k |
| Turn 10 | 87k |

Input mỗi request tăng $O(n)$; tổng input qua $n$ request tăng $O(n^2)$.

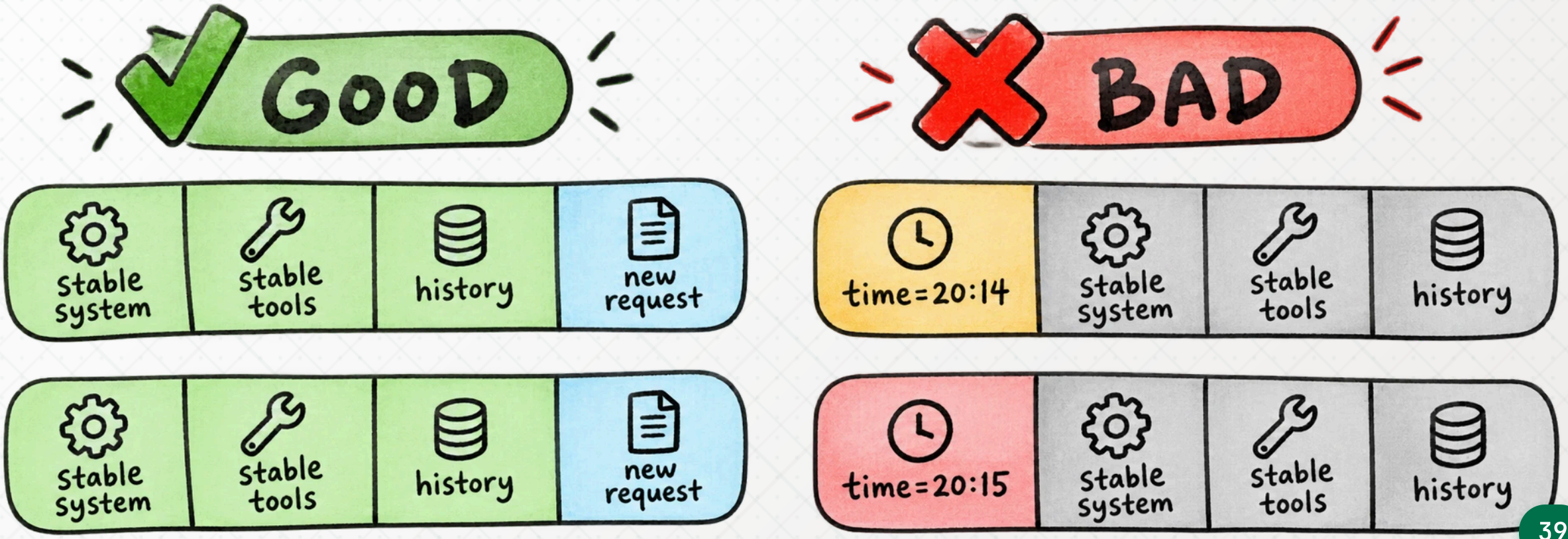
☕ Giả định: mỗi turn thêm lượng history *gần như cố định*

$$\text{input}_n \approx 51\text{k} + 4\text{k}\,(n-1)$$

$$\sum_{i=1}^{n} \text{input}_i = O(n^2)$$

# Prefix cache invalidation

> Thay đổi rất sớm trong prefix có thể làm **mất reuse** cache ở phần phía sau.



# Thói quen thân thiện với cache

- **Static prefix**, dynamic suffix
- Giữ **tool definitions** ổn định khi có thể
- Ưu tiên **append-only history** trong giai đoạn cần reuse
- Đo `cached_tokens`, cache write và cache read, *không* đoán

# Warm cache, cold cache và TTL

**Warm**: prefix có khả năng còn được tái sử dụng

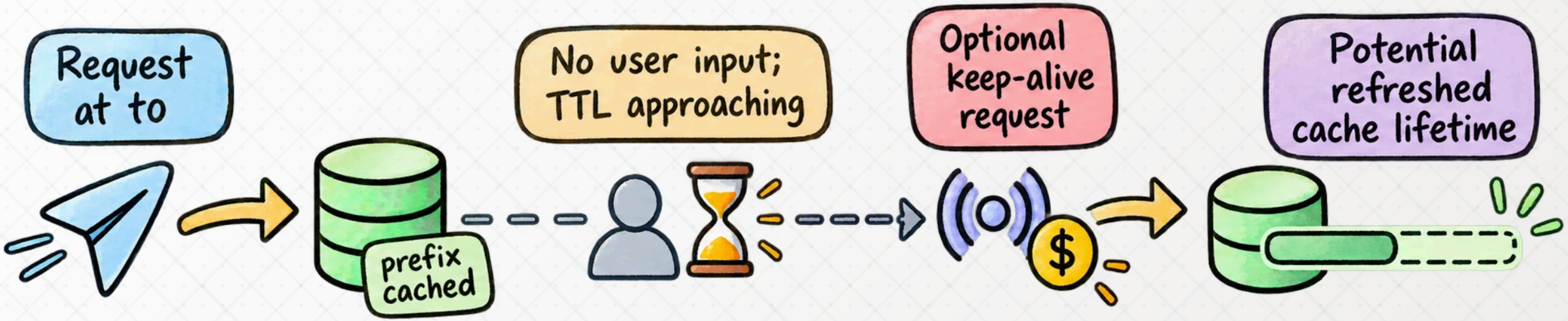
**Cold**: prefix không còn được tái sử dụng theo chính sách hiện hành

**TTL**: thời hạn hoặc điều kiện provider duy trì cache

Không có TTL hay chính sách write/read **chung cho mọi model**.

# Keep cache warm

Keep-alive request **vẫn tính phí**, và không chắc gia hạn được toàn bộ cache.



# Compaction và cache prefix

> Compaction có thể **phá cache prefix**.

- **Option 1**: giữ history 50k cached tokens
- **Option 2**: compact còn 12k tokens, nhưng prefix thay đổi
- Request kế tiếp: phương án nào rẻ hơn?
- Tính cả cache read/write, compaction call, output và các lượt sau

# Cache warming là khoản đầu tư

- Chi phí keep-alive xảy ra *ngay bây giờ*
- Lợi ích chỉ có *nếu* có request sau và cache reuse thành công
- Câu hỏi: **expected benefit** có lớn hơn keep-alive cost?

> Cache warming là đầu tư cho **reuse trong tương lai**.

# Công thức expected savings

$$\text{Expected savings} = P(\text{next request}) \times (C_{\text{cold}} - C_{\text{warm}})$$

$$\text{Keep warm} \iff \text{Expected savings} > \sum_{k} C_{\text{keep-alive},\, k}$$

☕ **Cộng thêm**: phí cache write, output, TTL policy, latency, quality

# Warm-cache affinity và model flexibility

So sánh model bằng **tổng chi phí** để đạt cùng chất lượng, không chỉ bằng giá trên bảng giá.

## Model A

- Đắt hơn
- Đang có **50k** cached tokens

## Model B

- Rẻ hơn
- *Chưa* có cache cho context này
- Đổi sang B phải **prefill lại** toàn bộ context

# Ba chiến lược

Chọn chiến lược có **chi phí kỳ vọng** thấp nhất, miễn đạt chất lượng và độ trễ.

| Strategy | Việc thực hiện | Đổi lại |
|---|---|---|
| **A: Warm A** | Keep-alive, dùng lại prefix A | Phí keep-alive |
| **B: Cold và Compact** | Compact rồi dùng A hoặc B | Phí compact, mất thông tin |
| **C: Switch B** | Đổi model, prefill lại từ đầu | Chất lượng của B |

# Dữ liệu mô phỏng

| USD / 1M tokens | Model A | Model B |
|---|---|---|
| Input | 5.00 | 1.00 |
| Cached input | 0.50 | 0.10 |
| Output | 20.00 | 5.00 |
| Cached prefix hiện tại | 50k | 0 |

Next request: **2k** input mới và **1k** output

# Tính thử một request

Chưa tính phí **keep-alive**, compaction, retry hoặc latency.

$C_B - C_A = \$0.002$: keep-alive phải rẻ hơn mức này mới có lời cho một lượt.

## A: warm, cache hit

$$\begin{aligned} C_A &= 50\text{k} \times \$0.50/\text{M} \\ &\quad + 2\text{k} \times \$5/\text{M} \\ &\quad + 1\text{k} \times \$20/\text{M} \\ &= \$0.055 \end{aligned}$$

## B: cold

$$\begin{aligned} C_B &= 52\text{k} \times \$1/\text{M} \\ &\quad + 1\text{k} \times \$5/\text{M} \\ &= \$0.057 \end{aligned}$$

# Challenge C: cache-aware model router

> Mục tiêu: chọn **warm, compact hoặc switch** cho request kế tiếp với chi phí kỳ vọng thấp nhất.

- ☕ Dùng bảng giá ở slide Dữ liệu mô phỏng, *không* cần gọi API
- ☕ Kịch bản: user quay lại sau **3 phút**, sau **7 phút**, **60%** quay lại trong 10 phút
- ☕ **Chứng minh**: quyết định cho từng kịch bản và khi nào quyết định đổi

**Timebox**: 30 phút.

# Challenge C: cache-aware model router

## Gợi ý

- Viết **chi phí kỳ vọng** của từng chiến lược theo các tham số
- Thời gian chờ vượt TTL: cần bao nhiêu lần keep-alive?
- Bắt đầu từ slide Tính thử một request: đã có chi phí A warm, B cold
- Đổi xác suất quay lại và retry rate để tìm **điểm đổi quyết định**

# Q & A